SOMNGUARD

Documento de Arquitectura

Autor: Equipo SomnGuard **Estado:** Estable **Fecha:** 2026-08-19 **Última actualización:** 2026-08-19

> **Actualización de stack (2026-08-16):** el backend se migra de C#/.NET a **Java 21 con Spring Boot 4.1.1 (Maven)**. Ver ADR-001: Backend en Java Spring Boot.
>
> **Actualización de arquitectura (2026-08-19):** el backend se estructura con **arquitectura hexagonal (puertos y adaptadores)** dentro del monolito modular. Ver ADR-002: Arquitectura hexagonal en el backend.

1. Propósito

Este documento describe la arquitectura general de SomnGuard, cubriendo la plataforma web, la app móvil, el backend, el dispositivo Raspberry Pi con cámara, la base de datos y la sincronización entre componentes.

Su objetivo es dejar trazada una vista técnica clara del sistema para guiar el desarrollo, la integración y las decisiones posteriores.

2. Alcance

SomnGuard se concibe como un sistema integral de software y hardware para monitoreo de somnolencia y fatiga al volante. La arquitectura incluye:

- Plataforma web para descarga de la app, acceso de administradores y dashboard de consulta.
- App móvil para usuario final.
- Backend en Java 21 (Spring Boot 4.1.1, Maven) como API principal.
- Dispositivo Raspberry Pi con cámara y software local de captura y visión.
- Base de datos PostgreSQL administrada con Liquibase.
- Servicio externo o independiente para almacenamiento de evidencia multimedia.

Quedan fuera de este documento las implementaciones detalladas de cada módulo, que se documentan en archivos técnicos o funcionales complementarios.

3. Contexto del sistema

SomnGuard busca detectar patrones de fatiga, somnolencia y microsueños para generar alertas preventivas y registrar evidencia asociada a eventos críticos. El sistema combina captura en edge, procesamiento local, sincronización hacia servidor y consulta desde plataforma web y móvil.

3.1 Actores principales

- Usuario final o conductor.
- Administrador de plataforma.
- Dispositivo SomnGuard instalado en campo.
- Backend central.
- Servicio de almacenamiento multimedia.

3.2 Flujo general

1. El dispositivo captura datos y evidencia desde la cámara.
2. El software local analiza o prepara la información para envío.
3. El backend recibe eventos, metadatos y referencias a evidencia.
4. La base de datos guarda la información estructurada.
5. La web y la app móvil consultan información y muestran alertas o dashboard.

4. Vista arquitectónica general

SomnGuard se organiza como una arquitectura híbrida edge-cloud con un backend modular.

- En el edge, el dispositivo Raspberry Pi ejecuta captura y lógica local básica.
- En el servidor, una API en Java (Spring Boot) centraliza reglas, autenticación, eventos, alertas y consultas.
- La persistencia estructurada vive en PostgreSQL.
- La evidencia multimedia se almacena fuera de la base de datos y se referencia mediante URL o identificador.

La forma de trabajo recomendada para el backend es un monolito modular con principios de clean architecture, de modo que cada dominio quede separado por responsabilidad sin fragmentar demasiado el despliegue.

```
[Device: Raspberry Pi] --- (sync/API) ---> [API Spring Boot] ---> [PostgreSQL]
        |
        +--- (multimedia) --------------> [Multimedia Storage]

[Web Client]
[Mobile Client] --> (API) --> [API Spring Boot]
```

5. Estilo de arquitectura

5.1 Backend

El backend se define como un monolito modular en Java 21 con Spring Boot 4.1.1, organizado por dominios funcionales con arquitectura hexagonal (puertos y adaptadores) por módulo (ver ADR-002).

5.2 Frontend

Se contemplan dos clientes:

- Web en React JS.
- Móvil en React Native.

5.3 Edge

El dispositivo Raspberry Pi funciona como nodo de captura y preprocesamiento. Su software local debe ser capaz de operar aunque exista conectividad limitada o nula.

6. Componentes principales

6.1 Plataforma web

La plataforma web cumple dos funciones principales:

- Página de acceso para descarga de la app móvil.
- Login y dashboard para administradores.

6.2 App móvil

La app móvil está orientada al usuario final y a la experiencia de consulta o recepción de información del sistema.

6.3 API backend

La API centraliza:

- Autenticación y autorización.
- Gestión de dispositivos asociados.
- Registro de eventos y evidencias.
- Seguimiento de alertas.
- Consulta de catálogos y parametrización.

6.4 Dispositivo Raspberry Pi

El dispositivo concentra:

- Captura desde cámara.
- Lógica local de visión o preparación de datos.
- Buffer local para operación offline.
- Sincronización posterior con el servidor.

6.5 Persistencia

La base de datos PostgreSQL almacena entidades estructuradas, catálogos, usuarios, dispositivos, eventos, alertas y referencias a evidencia multimedia.

6.6 Almacenamiento multimedia

La evidencia multimedia no se guarda directamente en PostgreSQL. Se almacena en un servicio aparte y la base de datos conserva solo la referencia, URL o identificador asociado.

7. Módulos funcionales del backend

Los módulos funcionales del backend siguen el catálogo de módulos (09-modules):

7.1 security

Responsable de cuentas, inicio de sesión, roles, permisos y auditoría de accesos.

7.2 parameterization

Responsable de los catálogos configurables: categoría de evento, severidad, tipo de medio, patrón de sonido y tipo de evento.

7.3 device-management

Responsable de asociar, desasociar y configurar dispositivos.

7.4 telemetry-service

Responsable de registrar eventos, evidencia y alertas desde el dispositivo.

7.5 monitoring

Responsable de notificaciones asociadas a eventos críticos y su seguimiento.

7.6 analytics

Responsable del módulo analítico: línea de tiempo de eventos, métricas de comportamiento, resumen descriptivo con IA y generación de reportes.

8. Backend: clean architecture con puertos y adaptadores (hexagonal)

La API se estructura con arquitectura limpia formalizada como **hexagonal (puertos y adaptadores)** por módulo, dentro del monolito modular. Cada módulo es un hexágono aislado que solo se comunica con otros módulos a través de sus puertos de entrada.

Reglas de dependencia:

- El dominio (`model` + `service`) está en el centro y no conoce Spring, JPA, HTTP ni la base de datos.
- `application/port/in` define los **puertos de entrada**: interfaces de casos de uso (lo que el módulo puede hacer).
- `application/port/out` define los **puertos de salida**: interfaces que el dominio necesita (repositorios, almacenamiento, notificadores, clientes externos).
- `application/usecase` implementa los casos de uso.
- `adapter/in` implementa la entrada: controladores REST (`web`) y consumidores de mensajes (`amqp`, si aplica).
- `adapter/out` implementa la salida: persistencia (`persistence`), almacenamiento multimedia (`storage`) y otros servicios externos.
- `platform` contiene lo transversal del backend (manejo de errores, logging y observabilidad) y **no va dentro de cada módulo**: es un paquete único al nivel de `com.somnguard`, junto a los módulos. Los módulos pueden depender de `platform`; `platform` no depende de ningún módulo.

En Java (Spring Boot) la organización típica por módulo es:

```
com.somnguard.<modulo>/
├── application/
```

```
│   ├── port/
│   │   ├── in/            # Interfaces de casos de uso (IngestEventUseCase, GetEventQuery, ...)
│   │   └── out/           # Interfaces de repositorios y servicios (EventRepository, ...)
│   └── usecase/           # Implementaciones de casos de uso (IngestEventService, ...)
├── domain/
│   ├── model/             # Entidades de negocio (Event, Evidence, AlertLog, ...)
│   └── service/           # Servicios de dominio (evaluación de severidad, validación, ...)
└── adapter/
    ├── in/
    │   ├── web/           # Controladores REST
    │   └── amqp/          # Consumidores de mensajes (si aplica)
    └── out/
        ├── persistence/   # Adaptadores JPA/PostgreSQL
        └── storage/       # Adaptadores de almacenamiento multimedia (MinIO/S3)
```

`platform` **no va dentro de cada módulo**: es un paquete único al nivel de `com.somnguard`, junto a los módulos, y todos lo comparten:

```
com.somnguard/
├── security/
├── parameterization/
├── device_management/
├── telemetry_service/
├── monitoring/
├── analytics/
└── platform/              # Transversal, fuera de los módulos: errores, logging, observabilidad
```

Los módulos pueden depender de `platform`; `platform` no depende de ningún módulo.

> **Convención de nombres:** en el catálogo y en la documentación los módulos se nombran en kebab-case (`device-management`, `telemetry-service`); en los **paquetes Java** se usa snake_case (`com.somnguard.device_management`, `com.somnguard.telemetry_service`), porque los guiones no son válidos en identificadores de paquete.

8.1 Beneficios esperados

- Separación de responsabilidades y regla de dependencia explícita hacia el dominio.
- Testabilidad: los casos de uso se prueban con mocks de los puertos, sin base de datos.
- Cambio tecnológico aislado: sustituir PostgreSQL, el almacenamiento multimedia o el transporte solo afecta a los adaptadores.
- Fronteras claras entre módulos ("monolito modular"): un módulo solo usa los puertos de entrada de otro.
- Camino evolutivo a microservicios si el proyecto lo requiere: cada módulo ya está aislado por sus puertos.

9. Datos y persistencia

9.1 Base de datos

La persistencia estructurada usa PostgreSQL.

9.2 Migraciones

Se emplea Liquibase para controlar cambios de esquema y versionado de base de datos.

La estructura prevista por carpetas para scripts de base de datos puede organizarse así:

```
somnguard-db/
├── 01_ddl/
│   ├── 00_extensions/
│   ├── 01_schemas/
│   ├── 02_types/
│   ├── 03_tables/
│   ├── 04_alter/
│   ├── 05_views/
│   ├── 06_materialized_views/
│   ├── 07_functions/
│   ├── 08_procedures/
│   ├── 09_triggers/
│   └── 10_indexes/
├── 02_dml/
│   ├── 00_inserts/
│   ├── 01_updates/
│   ├── 02_deletes/
│   ├── 03_upserts/
│   └── 04_patches/
├── 03_dcl/
│   ├── 00_roles/
│   ├── 01_grants/
│   └── 02_policies/
├── 04_tcl/
│   ├── 00_transaction_blocks/
│   └── 01_manual_recoveries/
└── 05_rollbacks/
        ├── 01_ddl/
        ├── 02_dml/
        ├── 03_dcl/
        └── 04_tcl/
```

Cada carpeta agrupa scripts relacionados por tipo de operación y facilita su mantenimiento, ejecución y reversión.

9.3 Entidades principales

La arquitectura de datos se alinea con los módulos ya definidos en el proyecto y con el modelo vigente (06-data-architecture):

- **security:** user, role, module, feature, role_feature, user_role, password_reset_request, audit_login.
- **parameterization:** event_category, severity, media_type, sound_pattern, event_type.
- **device-management:** device, device_assignment, device_config.
- **telemetry-service:** event, evidence, alert_log.
- **monitoring:** notification.
- **analytics:** vistas/reportes derivados (sin entidades transaccionales).

10. Autenticación y autorización

El mecanismo de autenticación y autorización está decidido: JWT RS256 para usuarios (registro, inicio y cierre de sesión, gestión de perfil, asignación de roles y permisos por funcionalidad/módulo) y API keys para dispositivos. El detalle contractual está en `../07-api-design/authentication.md` (ver ADR-001 y QR-003).

La arquitectura debe permitir distinguir al menos entre usuario final y administrador de plataforma, aunque el modelo de roles pueda crecer después.

11. Sincronización entre edge y servidor

El dispositivo debe operar en modo offline cuando no exista conectividad.

11.1 Comportamiento esperado

- Captura local de eventos y evidencia.
- Almacenamiento temporal en el dispositivo.
- Reintento de sincronización cuando vuelva la conexión.
- Envío de datos estructurados en JSON y de archivos multimedia al backend o a su servicio de almacenamiento asociado.

11.2 Consideraciones técnicas

- Validación de integridad antes de enviar.
- Identificadores consistentes entre eventos y evidencia.
- Tolerancia a pérdida temporal de red.
- Evitar duplicados en la recepción.

12. Alertas y notificaciones

La arquitectura debe soportar alertas preventivas y notificaciones asociadas a eventos críticos.

12.1 Tipos de respuesta

- Alerta local en el dispositivo.
- Notificación push en la app móvil, con o sin la aplicación abierta.
- Seguimiento del estado de la notificación.

13. Despliegue

La solución se plantea como una arquitectura híbrida edge + servidor.

- El dispositivo Raspberry Pi queda en campo como nodo edge.
- El backend y la base de datos se despliegan en infraestructura centralizada, local o en nube según la decisión operativa posterior.
- La evidencia multimedia vive en un almacenamiento externo o especializado.

13.1 Vista de despliegue

- Cliente web: navegador.
- Cliente móvil: Android/iOS con React Native.
- Edge: Raspberry Pi + cámara.
- Servidor: API Java (Spring Boot).
- Datos: PostgreSQL.
- Multimedia: servicio de archivos u objeto externo.

14. Integración continua y despliegue continuo

Aunque el detalle de operación puede documentarse en otro archivo, esta arquitectura sí considera integración continua como parte del sistema.

14.1 Intención

- Validar cambios de código de forma automática.
- Facilitar despliegues controlados.
- Reducir errores de integración.

14.2 Herramienta esperada

- GitHub Actions, si se mantiene la línea ya definida en la documentación del proyecto.

15. Observabilidad y auditoría

La arquitectura debe prever trazabilidad básica de la operación del sistema.

15.1 Elementos mínimos

- Logs técnicos del backend.
- Registro de eventos recibidos.
- Registro de alertas generadas.
- Trazabilidad de sincronización entre dispositivo y servidor.

16. Respaldo y recuperación

La arquitectura debe contemplar respaldo básico de:

- Base de datos.
- Evidencias asociadas a eventos críticos.
- Configuración de dispositivos, cuando aplique.

También debe existir una estrategia de recuperación ante fallos de red, fallos de sincronización o pérdida temporal de conectividad.

17. Requisitos no funcionales

Los requisitos no funcionales principales que debe soportar la arquitectura son:

- Seguridad.
- Mantenibilidad.
- Tolerancia a fallos.
- Baja latencia en alertas críticas.
- Escalabilidad razonable.
- Trazabilidad de eventos.
- Privacidad de datos sensibles.

18. Restricciones y supuestos

La arquitectura parte de los siguientes supuestos:

- El proyecto usa Java 21, Python, PostgreSQL, React JS y React Native.

- El hardware principal es Raspberry Pi con cámara.
- La captura y parte del procesamiento ocurren en edge.
- La persistencia multimedia no se hará en la base de datos principal.
- La nube o infraestructura final puede ajustarse más adelante

---

Vigencia: A partir del 2026-08-19